ST. VINCENT PALLOTTI
COLLEGE OF ENGINEERING & TECHNOLOGY, NAGPUR
AN AUTONOMOUS INSTITUTION

Approved by the AICTE (F.No.06/07/MS/ENGG/2004/029), DTE(2NGC/2004/Mantralaya/7) &
Affiliated to Nagpur University (NAVI/MVI/04/694/S/158)
(Managed by The Nagpur Pallottine Society)
Accredited by NAAC with 'A' Grade Post Box No. 128, G.P.O. Nagpur 440001

DEPARTMENT OF COMPUTER SCIENCE & BUSINESS SYSTEMS

Assignment 1

Course: Design and Analysis of Algorithms
Course Code: 24CB501T
Semester: 5th
Year: 3rd

UID: 23016027
Student Name: Shravan Vijay Sapate
Faculty Name: Prof. Pradnya S. Moon

Title: Case Study – FRIEND SUGGESTIONS USING GRAPH CLUSTERING

Objective

The main objective of this case study is to enable students to understand the practical relevance of algorithmic concepts by analyzing their application in real-world systems. Specifically, this study focuses on how **Graph Clustering Algorithms** can be used to design intelligent **Friend Suggestion Systems** used in popular social media platforms.
Through this study, students learn how theoretical algorithmic approaches, such as community detection, can be applied to large-scale social networks to improve connectivity and user engagement.

Expected Learning Outcomes

After completing this assignment, students will be able to:

1. Understand and explain how graph-based algorithms are applied in real-world systems such as social networks.
2. Analyze the efficiency, scalability, and performance of community detection algorithms using asymptotic notations.

3. Relate theoretical algorithmic principles such as *greedy methods*, *graph traversal*, and *optimization* to real-time engineering problems.
4. Develop analytical and research-based writing skills for interpreting algorithmic performance.
5. Gain exposure to handling real-world data represented as complex networks.

---

1. Problem Definition

1.1 Background and Real-World Context

Social media platforms like Facebook, Instagram, and LinkedIn use sophisticated algorithms to suggest friends or connections. These recommendations help users expand their networks based on mutual friends, common interests, or shared communities.
In computer science, such relationships can be modeled as a **Graph**, where:

- Each **node (vertex)** represents a user.
- Each **edge** represents a friendship or an interaction between two users.

When a new user joins or an existing user interacts with others, the platform continuously analyzes patterns of connectivity to identify groups (or clusters) of users who share similar characteristics or mutual connections.

1.2 Problem Statement

Given a social graph **G(V, E)** where **V** is the set of users and **E** is the set of friendships between them, the goal is to suggest potential friends to a user **u ∈ V** who are **not directly connected** to **u**, but belong to the same **cluster** (community) as **u**.

Formally, if users **u** and **v** belong to the same cluster **C**, and there exists no edge *(u, v)* in **E**, then **v** can be recommended to **u** as a potential friend.

1.3 Challenges in the Problem

- The **social graph** is highly dynamic and may contain millions of nodes and edges.
- Determining communities efficiently requires **scalable algorithms**.
- The **graph density** (number of edges) increases quadratically with users, making brute-force solutions infeasible.
- **Overlapping communities** (users belonging to multiple interest groups) make cluster detection complex.
- Balancing **accuracy vs. computational cost** is crucial in real-time friend suggestion systems.

---

2. Algorithm Explanation

2.1 Concept of Graph Clustering

Graph clustering (also known as **community detection**) involves partitioning a graph into groups of nodes (clusters) such that nodes within the same cluster are highly interconnected, while connections between clusters are sparse.
This approach helps uncover **hidden community structures** in complex networks like social media.

2.2 Techniques Used

The Friend Suggestion algorithm relies on **community detection** methods, particularly:

1. **Louvain Modularity Optimization** – a greedy optimization algorithm that maximizes modularity.
2. **Label Propagation Algorithm (LPA)** – assigns labels to nodes that propagate through their neighbors to form communities.

3. **Spectral Clustering** – uses eigenvalues of graph Laplacian matrices to group similar nodes.

2.3 Step-by-Step Working

1. **Graph Construction**
   Represent all users and their friendships as a graph **G(V, E)** using an adjacency matrix or list.
2. **Calculate Similarity Matrix**
   Measure similarity between users based on mutual friends, interaction frequency, or shared interests.
3. **Apply Clustering Algorithm**
   Use Louvain or LPA to divide users into communities by optimizing modularity or label similarity.
4. **Identify Potential Connections**
   For each user **u**, identify nodes in the same cluster that are not already connected to **u**.
5. **Generate Recommendations**
   Rank the unconnected users by their similarity or degree of common neighbors, and suggest the top candidates.

2.4 Example

Consider the following network of users:
A, B, C, D, E, F.

Edges:
(A–B), (B–C), (C–D), (E–F).

Here, (A, B, C, D) form a dense subgraph → **Cluster 1**, and (E, F) form **Cluster 2**.
If user A is not connected to D, but both belong to the same cluster, **D** can be suggested to **A** as a new friend.

---

3. Inputs and Outputs

Parameter	Description
Input	Social graph (nodes: users; edges: friendships or interactions)
Process	Cluster users based on graph connectivity and shared neighbors
Output	Suggested friend list for each user

3.1 Input Data Representation

The input graph can be stored using:

- **Adjacency Matrix** – suitable for dense graphs.
- **Adjacency List** – efficient for sparse graphs.

For example:

A: B, C

B: A, C, D

C: A, B, D

D: B, C

3.2 Output Example

Friend Suggestions:

A → D

B → (none)

C → (none)

D → A

---

4. Performance Analysis

4.1 Time Complexity

- Graph Construction: **O(V + E)**

- Clustering (Louvain): **O(n log n)**
- Recommendation Generation: **O(V^2)** in the worst case.

4.2 Space Complexity

- Adjacency List: **O(V + E)**
- Adjacency Matrix: **O(V^2)**

4.3 Efficiency

The algorithm efficiently handles large social graphs using modularity-based clustering. It scales well for networks with millions of nodes when implemented using distributed systems (e.g., Apache Spark GraphFrames).

4.4 Limitations

- High computational cost for real-time updates.
- Accuracy may reduce in sparse or noisy graphs.
- Dynamic graph changes require periodic recalculation.

---

5. Conclusion

The **Friend Suggestion System using Graph Clustering** effectively demonstrates how advanced algorithmic concepts are applied in real-world social platforms.
By grouping users into communities, the algorithm enhances the relevance of suggestions, thereby increasing user engagement.
The study bridges theoretical learning (graph theory and clustering) with practical application (social networking).

Future Scope:

- Integration with **Machine Learning** models for personalized ranking.
- Considering **edge weights** for stronger recommendations.
- Implementation on distributed computing frameworks for scalability.

---

6. Class of the Problem (P / NP / NP-Complete / NP-Hard)

The **Community Detection Problem** belongs to the **NP-Hard** class.
Finding an exact modularity-maximizing clustering requires exploring exponentially many partitions.
However, heuristic and greedy algorithms like the **Louvain method** approximate optimal solutions in polynomial time, making them practical for large datasets.

References (APA Style)

1. Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). *Fast unfolding of communities in large networks.* Journal of Statistical Mechanics.
2. Fortunato, S. (2010). *Community detection in graphs.* Physics Reports, 486(3–5), 75–174.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd ed.).* MIT Press.
4. NetworkX Documentation. (2024). *Graph clustering and community detection algorithms.*
5. Leskovec, J., & Sosič, R. (2016). *SNAP: Stanford Network Analysis Platform.*